JPA Tutorial

Large Scale and Multi-Structured Data Bases – Workgroup Task 1

University of Pisa

Academic Year 2019-2020

Barsellotti Luca

Gholami Sina

Pacini Federico

Paolini Emilio

1 INDEX

2 JPA Introduction 2
3 Entities 3
3.1 Persist vs merge 4
4 Relationships 4
4.1 One to one 4
4.1.1 Foreing Key 5
4.1.2 Shared Primary Key 9
4.1.3 Join Table 11
4.2 OneToMany 14
4.3 ManyToMany 17
4.3.1 Basic approach 17
4.3.2 Composite Key 21
4.3.3 New Entity 25
5 Updating 29
6 Deleting 30
7 JPQL 30
8 Inheritance 31
8.1 Mapped Superclass 31
8.2 Table per class 32
8.3 Single table 33
8.4 Joined 34

2 JPA INTRODUCTION

The Java Persistence API (JPA) is a Java specification that links the relational database and the object-oriented programming. Java objects are modelled as JPA entities. There was a mismatch between object-oriented implementation and relational database concepts. So, new tools for Object-Relational Mapping (ORM) were born, as Hibernate, EclipseLink and iBatis. Hibernate, an ORM tool, is an implementation of JPA and it has some extra features which are not in JPA standards.

JPA is based on metadata annotations to define the mapping between objects and relational database. What to do with these objects is defined by Entity Manager. Then, to communicate to the database, a JDBC library is used.

3 ENTITIES

To generate an entity in JPA, we need to use the annotation `@Entity` before the declaration of the class. Each attribute in the class is an attribute of the generated entity, and the characteristics of the attribute can be defined by using the annotations.

Then, to insert a new row inside the table corresponding to that new entity, we need to use the EntityManager and EntityManagerFactory. The EntityManagerFactory allow us to connect to the database. The EnitityManager is used to manage the transaction (to open, execute operations on tables and close).

Example:

Let's create a student table, where each student has an ID, name, surname and date of birth. We can see the `@Entity` annotation before the class declaration and the `@Table(name="student")` annotation, that allows us to specify a different name for the table (otherwise, the table will have the class name). The `@Id` annotation, inserted before the id attribute, is used to specify the primary key of the table.

```
//Student.java

import javax.persistence.*;
import java.time.LocalDateTime;

@Entity
@Table(name = "student")
public class Student {

    @Id
    private long id;
    private String name;
    private String surname;
    private LocalDateTime birthDate;
    //getters and setters
}
```

In the following code, we are going to create the student entity manager class, that allows us to execute query on student table. The `setup` method creates an EntityManagerFactory object, the `exit` method closes this object and the `create` method inserts a new student object inside the table. Inside the transaction we can both use persist and merge. The first one inserts a persistence object and stop, the second one inserts a persistence object and, if necessary, complete the foreign attribute values.

```
//StudentEM.java

import javax.persistence.EntityManager;
import javax.persistence.EntityManagerFactory;
import javax.persistence.Persistence;

public class StudentEM {
    EntityManagerFactory factory;
    EntityManager entityManager;

    public void setup(){
```

```
        factory = Persistence.createEntityManagerFactory("university");
    }

    public void exit(){
        factory.close();
    }

    public void create(Student student){
        try{
            entityManager = factory.createEntityManager();
            entityManager.getTransaction().begin();
            entityManager.persist(student); //entityManager.merge(student)
            entityManager.getTransaction().commit();
        } catch (Exception e){
            e.printStackTrace();
        } finally {
            entityManager.close();
        }
    }
}
```

3.1 Persist vs merge

When we want to persist an entity object, we have two options: persist and merge. The persist method:

- Insert a new row in the database.
- Attach the object to the entity manager.

The merge method:

- Find an attached object with the same id and update it.
- If this object exists, update and return the already attached object.
- If doesn't exist, insert the new row in the database.

So, the main difference between these two methods is that persist inserts the new object and attaches it to the entity manager, merge inserts only if there isn't an attached object with the same id (in this case, update it) and doesn't attach the object to the entity manager. The main advantage of persist is to avoid mistakes in updating an object when we want to insert, throwing an EntityExistsException.

4 Relationships

4.1 One to one

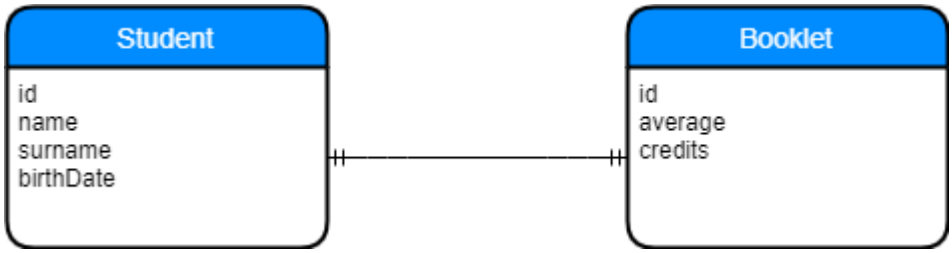

Figure 1: One to one relation example between a student and his booklet.

Let's assume that we have a University Database, where each student has a booklet. This is a one-to-one relation: the student ID identifies both a Student and a Booklet (figure 1).

We have three options to handle this.

4.1.1 Foreing Key

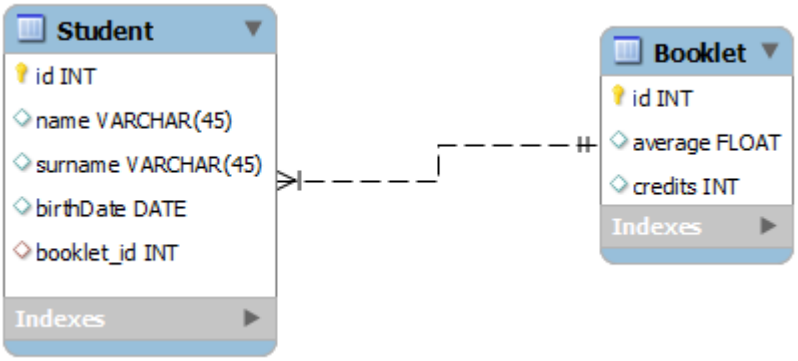

Figure 2: One to one relation with foreign key example, between a student and his booklet.

In the student entity, that has the foreign key, we need to add the `@OneToOne(cascade = CascadeType.ALL)` and `@JoinColumn(referencedColumnName = "id")` before a Booklet object attribute. The name of the referenced column must be the name used in the other entity.

```
//Student.java

import javax.persistence.*;
import java.time.LocalDateTime;

@Entity
@Table(name = "student")
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    @Column(name = "id")
    private long id;
    @OneToOne(cascade = CascadeType.ALL)
    @JoinColumn(referencedColumnName = "id")
    private Booklet booklet;
    private String name;
    private String surname;
    private LocalDateTime birthDate;

    public long getId(){
        return id;
    }

    public void setName(String name){
        this.name = name;
    }

    public void setId(long id){ this.id = id;}

    public void setSurname(String surname) {
        this.surname = surname;
```

```
    }

    public void setBirthDate(LocalDateTime birthDate) {
        this.birthDate = birthDate;
    }

    public void setBooklet(Booklet booklet) {
        this.booklet = booklet;
    }

    public String getName(){
        return name;
    }

    public String getSurname(){
        return surname;
    }

    public LocalDateTime getBirthDate() {
        return birthDate;
    }

    public Booklet getBooklet() {
        return booklet;
    }

    public String toString(){
        return "ID: "+this.getId()+" Name: "+this.getName()+" Surname:
"+this.getSurname();
    }
}
```

In the booklet entity, that has the referenced column, we need to add the `@OneToOne(mappedBy = “booklet”)` annotation before a Student object attribute. The name indicated in the annotation must be the name of the column used as foreign key in the other entity.

```
//Booklet.java

import javax.persistence.*;

@Entity
@Table(name = "booklet")
public class Booklet {

    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    @Column(name = "id")
    private long id;
    @OneToOne(mappedBy = "booklet")
    private Student student;
    private float average;
    private int credits;

    public void setId(long id) {
        this.id = id;
    }
```

```
    public void setAverage(float average) {
        this.average = average;
    }

    public void setCredits(int credits) {
        this.credits = credits;
    }

    public void setStudent(Student student) {
        this.student = student;
    }

    public long getId() {
        return id;
    }

    public float getAverage() {
        return average;
    }

    public int getCredits() {
        return credits;
    }

    public Student getStudent() {
        return student;
    }

    public String toString(){
        return "ID: "+this.getId()+" Average: "+this.getAverage()+" Credits:
"+this.getCredits();
    }
}
```

In the student entity manager class we need to use the merge function to persist a student object.

```
//StudentEM.java

import javax.persistence.EntityManager;
import javax.persistence.EntityManagerFactory;
import javax.persistence.Persistence;

public class StudentEM {
    EntityManagerFactory factory;
    EntityManager entityManager;

    public void setup(){
        factory = Persistence.createEntityManagerFactory("university");
    }

    public void exit(){
        factory.close();
    }

    public void create(Student student){
        try{
            entityManager = factory.createEntityManager();
```

```
            entityManager.getTransaction().begin();
            entityManager.merge(student);
            entityManager.getTransaction().commit();
        } catch (Exception e){
            e.printStackTrace();
        } finally {
            entityManager.close();
        }
    }
}
```

Instead, in the booklet entity manager we can simply use the persist function to persist the booklet object.

```
//BookletEM.java

import javax.persistence.EntityManager;
import javax.persistence.EntityManagerFactory;
import javax.persistence.Persistence;

public class BookletEM {
    EntityManagerFactory factory;
    EntityManager entityManager;

    public void setup(){
        factory = Persistence.createEntityManagerFactory("university");
    }

    public void exit(){
        factory.close();
    }

    public long create(Booklet booklet){
        long id = 0;
        try{
            entityManager = factory.createEntityManager();
            entityManager.getTransaction().begin();
            entityManager.persist(booklet);
            entityManager.getTransaction().commit();
            id = booklet.getId();
        } catch (Exception e){
            e.printStackTrace();
        } finally {
            entityManager.close();
        }
        return id;
    }
}
```

In this solution we used two different IDs: one for the student and one for the booklet. The booklet ID is inserted as foreign key in the student table. This is a bidirectional relationship. The ID is incremented twice: once for the student ID and once for the booklet ID.

4.1.2 Shared Primary Key

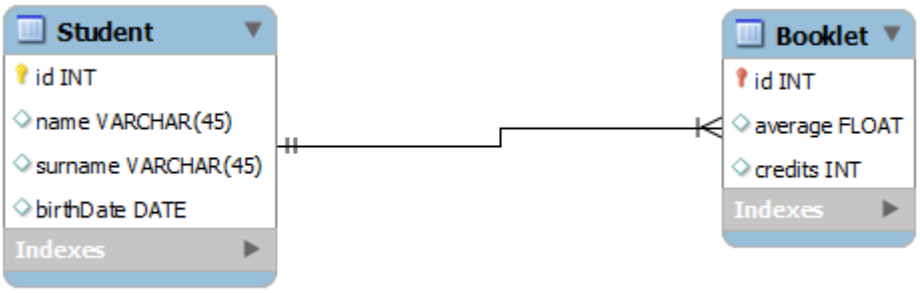

Figure 3: One to one relation with shared primary key example, between a student and his booklet.

In this solution, there is only the student ID and it is used as shared foreign key by the booklet. To do that, in the student entity, that has the original ID, we need to add the `@OneToOne(mappedBy="student", cascade=CascadeType.ALL)` annotation.

```
//Student.java

import javax.persistence.*;
import java.time.LocalDateTime;

@Entity
@Table(name = "student")
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id")
    private long id;
    @OneToOne(mappedBy = "student", cascade = CascadeType.ALL)
    private Booklet booklet;
    private String name;
    private String surname;
    private LocalDateTime birthDate;
    //getters and setters
}
```

in the booklet entity, that has the foreign key used as primary key, we need to add `@OneToOne` and `@MapsId` annotations.

```
//Booklet.java

import javax.persistence.*;

@Entity
@Table(name = "booklet")
public class Booklet {

    @Id
    @Column(name = "id")
    private long id;
    @OneToOne
    @MapsId
    private Student student;
```

```
    private float average;
    private int credits;
    //getters and setters
}
```

We may call the persist method in the student entity manager, but we need to use merge method in the booklet entity manager.

```
//StudentEM.java

import javax.persistence.EntityManager;
import javax.persistence.EntityManagerFactory;
import javax.persistence.Persistence;

public class StudentEM {
    EntityManagerFactory factory;
    EntityManager entityManager;

    public void setup(){
        factory = Persistence.createEntityManagerFactory("university");
    }

    public void exit(){
        factory.close();
    }

    public long create(Student student){
        long id=0;
        try{
            entityManager = factory.createEntityManager();
            entityManager.getTransaction().begin();
            entityManager.persist(student);
            entityManager.getTransaction().commit();
            id = student.getId();
        } catch (Exception e){
            e.printStackTrace();
        } finally {
            entityManager.close();
        }
        return id;
    }
}
```

```
//BookletEM.java

import javax.persistence.EntityManager;
import javax.persistence.EntityManagerFactory;
import javax.persistence.Persistence;

public class BookletEM {
    EntityManagerFactory factory;
    EntityManager entityManager;

    public void setup(){
        factory = Persistence.createEntityManagerFactory("university");
    }
```

```
    public void exit(){
        factory.close();
    }

    public long create(Booklet booklet){
        long id = 0;
        try{
            entityManager = factory.createEntityManager();
            entityManager.getTransaction().begin();
            entityManager.merge(booklet);
            entityManager.getTransaction().commit();
            id = booklet.getId();
        } catch (Exception e){
            e.printStackTrace();
        } finally {
            entityManager.close();
        }
        return id;
    }
}
```

4.1.3 Join Table

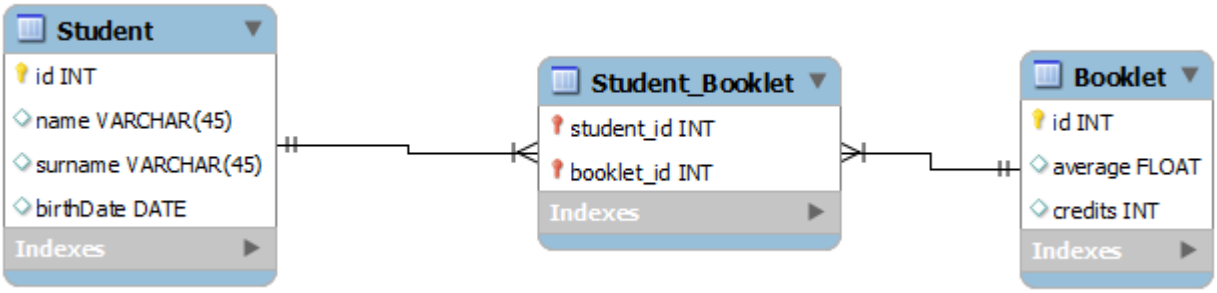

Figure 4: One to one relation using join table example, between a student and his booklet.

In this solution we use again (as we did in the foreing ket solution) two different ID: one for student and one for booklet. In one of the two entities, in this case the student, we have to insert `@OneToOne(cascade = CascadeType.ALL)` and `@JoinTable` annotations. In `@JoinTable` we have to explain how is this new entity is built: "`name`" to indicate the table name, "`joinColumns={@JoinColumn(name="student_id", referencedColumnName="id")}`" to indicate the link between student entity and student_booklet entity, "`inverseColumns={@JoinColumn(name="booklet_id", referencedColumnName="id")}`" to indicate the link between booklet entity and student_booklet entity. In the other entity, in our case the booklet entity, we have to insert `@OneToOne(mappedBy="booklet")` annotation.

```
//Student.java

import javax.persistence.*;
import java.time.LocalDateTime;

@Entity
@Table(name = "student")
public class Student {
```

```
    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    @Column(name = "id")
    private long id;
    @OneToOne(cascade = CascadeType.ALL)
    @JoinTable(name = "student_booklet", joinColumns = {
            @JoinColumn(name = "student_id", referencedColumnName = "id")
    }, inverseJoinColumns = {
            @JoinColumn(name = "booklet_id", referencedColumnName = "id")
    })
    private Booklet booklet;
    private String name;
    private String surname;
    private LocalDateTime birthDate;
    //getters and setters
}
```

```
//Booklet.java

import javax.persistence.*;

@Entity
@Table(name = "booklet")
public class Booklet {

    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    @Column(name = "id")
    private long id;
    @OneToOne(mappedBy = "booklet")
    private Student student;
    private float average;
    private int credits;
    //getters and setters
}
```

We may call the persist method in the booklet entity manager, but we need to use merge method in the student entity manager.

```
//StudentEM.java

import javax.persistence.EntityManager;
import javax.persistence.EntityManagerFactory;
import javax.persistence.Persistence;

public class StudentEM {
    EntityManagerFactory factory;
    EntityManager entityManager;

    public void setup(){
        factory = Persistence.createEntityManagerFactory("university");
    }

    public void exit(){
        factory.close();
    }
```

```java
    public long create(Student student){
        long id=0;
        try{
            entityManager = factory.createEntityManager();
            entityManager.getTransaction().begin();
            entityManager.merge(student);
            entityManager.getTransaction().commit();
            id = student.getId();
        } catch (Exception e){
            e.printStackTrace();
        } finally {
            entityManager.close();
        }
        return id;
    }
}
```

```java
//BookletEM.java

import javax.persistence.EntityManager;
import javax.persistence.EntityManagerFactory;
import javax.persistence.Persistence;

public class BookletEM {
    EntityManagerFactory factory;
    EntityManager entityManager;

    public void setup(){
        factory = Persistence.createEntityManagerFactory("university");
    }

    public void exit(){
        factory.close();
    }

    public long create(Booklet booklet){
        long id = 0;
        try{
            entityManager = factory.createEntityManager();
            entityManager.getTransaction().begin();
            entityManager.persist(booklet);
            entityManager.getTransaction().commit();
            id = booklet.getId();
        } catch (Exception e){
            e.printStackTrace();
        } finally {
            entityManager.close();
        }
        return id;
    }
}
```

4.2 OneToMany

Figure 5: One to many relation example between a booklet and the exams given by the booklet's student.

Let's assume that each Booklet is connected to all the exams given by the student. This is a one to many relationship.

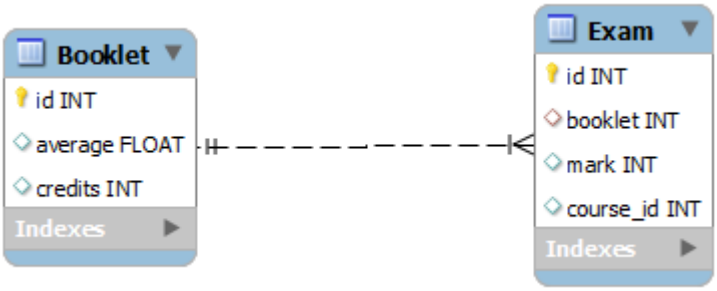

Figure 6: Implementation of a one to many relation example between a booklet and the exams given by the booklet's student.

In the booklet entity we have a `Set<Exam>` attribute that contains all the exams object related to that booklet and it needs an annotation `@OneToMany(mappedBy="booklet")`, where "booklet" is the name of the attribute inside exam that is connected to booklet.

```
//Booklet.java

import javax.persistence.*;
import java.util.Set;

@Entity
@Table(name = "booklet")
public class Booklet {

    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    @Column(name = "id")
    private long id;
    @OneToMany(mappedBy = "booklet")
    private Set<Exam> exams;
    private float average;
    private int credits;

    public void setId(long id) {
        this.id = id;
    }

    public void setAverage(float average) {
        this.average = average;
    }
```

```
    public void setCredits(int credits) {
        this.credits = credits;
    }

    public long getId() {
        return id;
    }

    public float getAverage() {
        return average;
    }

    public int getCredits() {
        return credits;
    }

    public Set<Exam> getExams() {
        return exams;
    }

    public String toString(){
        return "ID: "+this.getId()+" Average: "+this.getAverage()+" Credits:
"+this.getCredits();
    }
}
```

In the exam entity, we need a `@ManyToOne` and `@JoinColumn(name = “booklet_id”, nullable = false)` annotations.

```
//Exam.java

import javax.persistence.*;

@Entity
public class Exam {
    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private long id;
    @ManyToOne
    @JoinColumn(name="booklet_id", nullable = false)
    private Booklet booklet;
    private int mark;
    private long course_id;

    public Exam(){ }

    public Exam(Booklet booklet, int mark, long course_id){
        this.booklet = booklet;
        this.mark = mark;
        this.course_id = course_id;
    }
}
```

In the booklet entity manager class, that represents the “one” entity in the “one-to-many” relationship, we can use the persist method.

```java
//BookletEM.java

import javax.persistence.EntityManager;
import javax.persistence.EntityManagerFactory;
import javax.persistence.Persistence;

public class BookletEM {
    EntityManagerFactory factory;
    EntityManager entityManager;

    public void setup(){
        factory = Persistence.createEntityManagerFactory("university");
    }

    public void exit(){
        factory.close();
    }

    public long create(Booklet booklet){
        long id = 0;
        try{
            entityManager = factory.createEntityManager();
            entityManager.getTransaction().begin();
            entityManager.persist(booklet);
            entityManager.getTransaction().commit();
            id = booklet.getId();
        } catch (Exception e){
            e.printStackTrace();
        } finally {
            entityManager.close();
        }
        return id;
    }
}
```

In the exam entity manager class, that represents the “many” entity in the “one-to-many” relationship, we need to use the merge method.

```java
//ExamEM.java

import javax.persistence.EntityManager;
import javax.persistence.EntityManagerFactory;
import javax.persistence.Persistence;

public class ExamEM {
    EntityManagerFactory factory;
    EntityManager entityManager;

    public void setup(){
        factory = Persistence.createEntityManagerFactory("university");
    }

    public void exit(){
        factory.close();
    }

    public void create(Exam exam){
        try{
```

```
            entityManager = factory.createEntityManager();
            entityManager.getTransaction().begin();
            entityManager.merge(exam);
            entityManager.getTransaction().commit();
        } catch (Exception e){
            e.printStackTrace();
        } finally {
            entityManager.close();
        }
    }
}
```

4.3 MANYTOMANY

Figure 7: Many-to-many relationship example between courses and students.

Let's assume that each student can enrol in a course. Each student can enrol in different courses, but each course can be enrolled by different students.

4.3.1 Basic approach

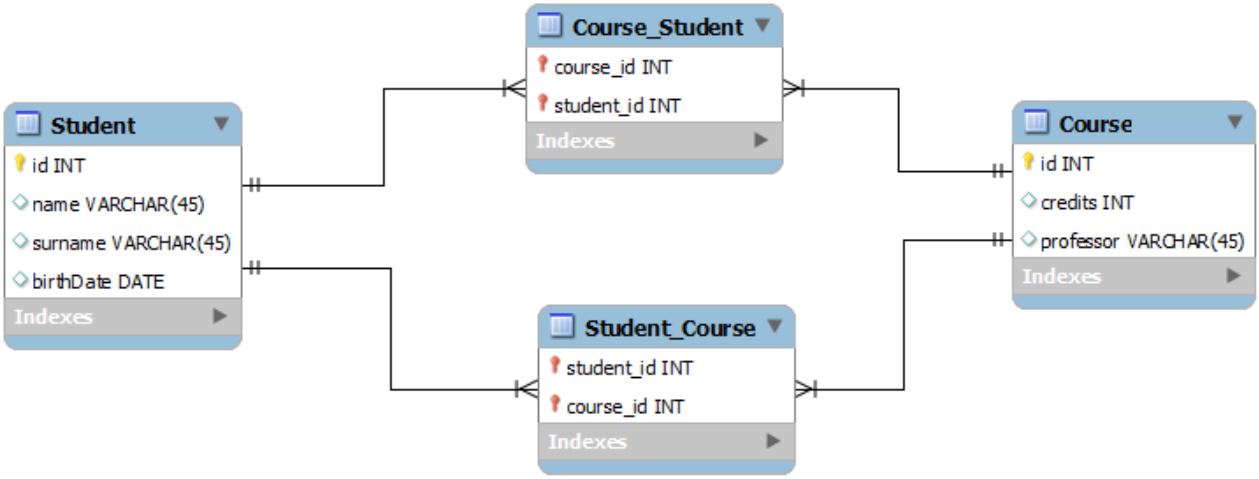

Figure 8: Implementation of the many-to-many relationship example between courses and students, using the basic approach.

In this solution we need to add a `@ManyToMany` annotation in both entities, a `Set<Course>` attribute in the student entity and a `Set<Student>` attribute in the course entity.

```java
//Student.java

import javax.persistence.*;
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.Set;

@Entity
@Table(name = "student")
public class Student {

    @Id
    private long id;
    @ManyToMany
    private Set<Course> courses;
    private String name;
    private String surname;
    private LocalDateTime birthDate;

    public Student(){}

    public Student(long id, String name, String surname, LocalDateTime birthDate){
        this.id = id;
        this.name = name;
        this.surname = surname;
        this.birthDate = birthDate;
    }

    public long getId(){
        return id;
    }

    public void setName(String name){
        this.name = name;
    }

    public void setId(long id){ this.id = id;}

    public void setSurname(String surname) {
        this.surname = surname;
    }

    public void setBirthDate(LocalDateTime birthDate) {
        this.birthDate = birthDate;
    }

    public Set<Course> getCourses() {
        return courses;
    }

    public String getName(){
        return name;
    }

    public String getSurname(){
        return surname;
    }

    public LocalDateTime getBirthDate() {
```

```
        return birthDate;
    }

    public void setCourses(Set<Course> courses) {
        this.courses = courses;
    }

    public String toString(){
        return "ID: "+this.getId()+" Name: "+this.getName()+" Surname:
"+this.getSurname();
    }
}
```

```
//Course.java

import javax.persistence.Entity;
import javax.persistence.Id;
import javax.persistence.ManyToMany;
import java.util.Set;

@Entity
public class Course {
    @Id
    private long id;

    @ManyToMany
    private Set<Student> students;

    private int credits;
    private String professor;

    public Course(){}
    public Course(long id, int credits, String professor, Set<Student> students){
        this.id = id;
        this.credits = credits;
        this.professor = professor;
        this.students = students;
    }

    public void setId(long id) {
        this.id = id;
    }

    public void setStudents(Set<Student> students) {
        this.students = students;
    }

    public void setCredits(int credits) {
        this.credits = credits;
    }

    public void setProfessor(String professor) {
        this.professor = professor;
    }

    public long getId() {
        return id;
    }
```

```
    public Set<Student> getStudents() {
        return students;
    }

    public int getCredits() {
        return credits;
    }

    public String getProfessor() {
        return professor;
    }
}
```

In one of the two entity manager classes we can use the persist method and in the other one we need to use the merge method. In the example, we put the merge method in the student entity manager class and the persist method in the course entity manager class. This means that we have to insert the courses and then the associated students.

```
//StudentEM.java

import javax.persistence.EntityManager;
import javax.persistence.EntityManagerFactory;
import javax.persistence.Persistence;

public class StudentEM {
    EntityManagerFactory factory;
    EntityManager entityManager;

    public void setup(){
        factory = Persistence.createEntityManagerFactory("university");
    }

    public void exit(){
        factory.close();
    }

    public long create(Student student){
        long id=0;
        try{
            entityManager = factory.createEntityManager();
            entityManager.getTransaction().begin();
            entityManager.merge(student);
            entityManager.getTransaction().commit();
            id = student.getId();
        } catch (Exception e){
            e.printStackTrace();
        } finally {
            entityManager.close();
        }
        return id;
    }
}
```

```
//CourseEM.java
```

```java
import javax.persistence.EntityManager;
import javax.persistence.EntityManagerFactory;
import javax.persistence.Persistence;

public class CourseEM {
    EntityManagerFactory factory;
    EntityManager entityManager;

    public void setup(){
        factory = Persistence.createEntityManagerFactory("university");
    }

    public void exit(){
        factory.close();
    }

    public void create(Course course){
        try{
            entityManager = factory.createEntityManager();
            entityManager.getTransaction().begin();
            entityManager.persist(course);
            entityManager.getTransaction().commit();
        } catch (Exception e){
            e.printStackTrace();
        } finally {
            entityManager.close();
        }
    }
}
```

4.3.2 Composite Key

This solution is useful when the relationship has an attribute. For example, let's assume that a student can assign a rating to a course. So, what we have is

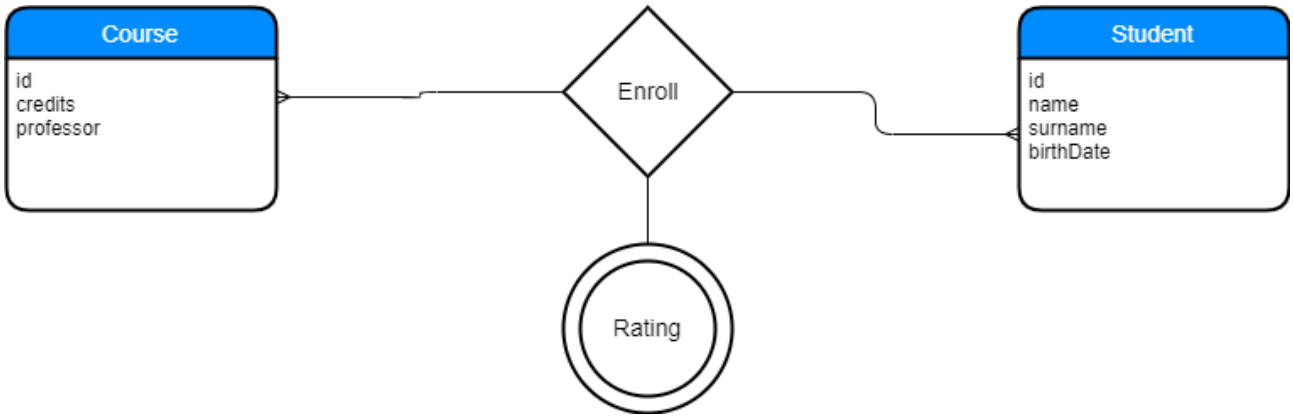

Figure 9: Representation of the many-to-many relationship example between courses and students, using a composite key.

With JPA we had the advantage that we could make only the Student and the Course entities without making connection tables. But without other entities or objects, there is no way to insert the value of the rating. So, now we need to insert two new classes: one for the composite key and one for the course rating entity.

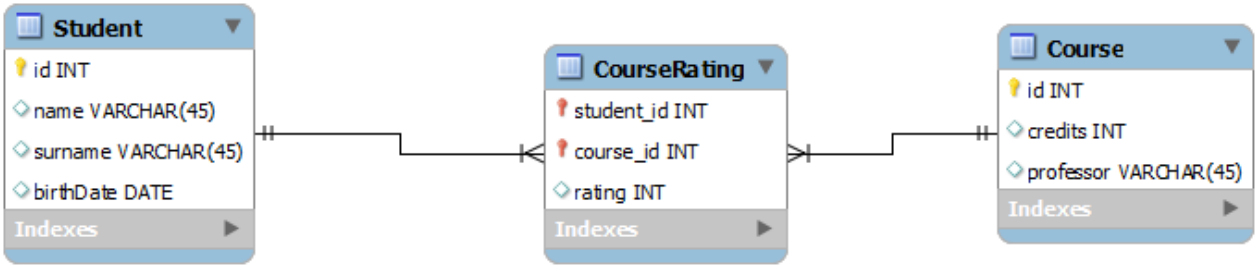

Figure 10: Implementation of the many-to-many relationship example between courses and students, using the composite key.

The relationships between student, course and the third class about course ratings are handled by using the OneToMany format (Chapter 3.2). Both student entity class and course entity class have a set of course rating entity objects as attribute with `@OneToMany` annotation. The course rating entity class, instead, has a course rating key object as id using `@EmbeddedId` annotation: the class of that object must implement Serializable and must have the `@Embeddable` annotation before declaration. Also, this class must override hashCode and equals methods.

```
//Student.java

import javax.persistence.*;
import java.time.LocalDateTime;
import java.util.Set;

@Entity
@Table(name = "student")
public class Student {

    @Id
    private long id;
    @OneToMany(mappedBy = "student")
    private Set<CourseRating> ratings;
    private String name;
    private String surname;
    private LocalDateTime birthDate;
    //getters and setters
}
```

```
//Course.java

import javax.persistence.Entity;
import javax.persistence.Id;
import javax.persistence.OneToMany;
import java.util.Set;

@Entity
public class Course {
    @Id
    private long id;

    @OneToMany(mappedBy = "course")
    private Set<CourseRating> ratings;
```

```
    private int credits;
    private String professor;
    //getters and setters
}
```

```
//CourseRatingKey.java

import javax.persistence.Column;
import javax.persistence.Embeddable;
import java.io.Serializable;

@Embeddable
public class CourseRatingKey implements Serializable {
    @Column
    private long student_id;

    @Column
    private long course_id;

    public CourseRatingKey(){
        student_id = 0;
        course_id = 0;
    }

    public CourseRatingKey(long student_id, long course_id){
        this.student_id = student_id;
        this.course_id = course_id;
    }

    @Override
    public int hashCode() {
        return super.hashCode();
    }

    @Override
    public boolean equals(Object obj) {
        return super.equals(obj);
    }
}
```

```
//CourseRating.java

import javax.persistence.*;

@Entity
public class CourseRating {
    @EmbeddedId
    private CourseRatingKey id;

    @ManyToOne
    @MapsId("student_id")
    @JoinColumn(name = "student_id")
    private Student student;

    @ManyToOne
    @MapsId("course_id")
    @JoinColumn(name = "course_id")
```

```
    private Course course;

    private int rating;
    //getters and setters
}
```

In this solution we used the merge method for all the three entity manager classes (student, course and course rating).

```
//StudentEM.java

import javax.persistence.EntityManager;
import javax.persistence.EntityManagerFactory;
import javax.persistence.Persistence;

public class StudentEM {
    EntityManagerFactory factory;
    EntityManager entityManager;

    public void setup(){
        factory = Persistence.createEntityManagerFactory("university");
    }

    public void exit(){
        factory.close();
    }

    public long create(Student student){
        long id=0;
        try{
            entityManager = factory.createEntityManager();
            entityManager.getTransaction().begin();
            entityManager.merge(student);
            entityManager.getTransaction().commit();
            id = student.getId();
        } catch (Exception e){
            e.printStackTrace();
        } finally {
            entityManager.close();
        }
        return id;
    }
}
```

```
//CourseEM.java

import javax.persistence.EntityManager;
import javax.persistence.EntityManagerFactory;
import javax.persistence.Persistence;

public class CourseEM {
    EntityManagerFactory factory;
    EntityManager entityManager;

    public void setup(){
        factory = Persistence.createEntityManagerFactory("university");
    }
```

```
    public void exit(){
        factory.close();
    }

    public void create(Course course){
        try{
            entityManager = factory.createEntityManager();
            entityManager.getTransaction().begin();
            entityManager.merge(course);
            entityManager.getTransaction().commit();
        } catch (Exception e){
            e.printStackTrace();
        } finally {
            entityManager.close();
        }
    }
}
```

```
//CourseRatingEM.java

import javax.persistence.EntityManager;
import javax.persistence.EntityManagerFactory;
import javax.persistence.Persistence;

public class CourseRatingEM {
    EntityManagerFactory factory;
    EntityManager entityManager;

    public void setup(){
        factory = Persistence.createEntityManagerFactory("university");
    }

    public void exit(){
        factory.close();
    }

    public void create(CourseRating rating){
        try{
            entityManager = factory.createEntityManager();
            entityManager.getTransaction().begin();
            entityManager.merge(rating);
            entityManager.getTransaction().commit();
        } catch (Exception e){
            e.printStackTrace();
        } finally {
            entityManager.close();
        }
    }
}
```

4.3.3 New Entity

Let's assume that, between each student and course there are multiple rows because a student can rate a single lecture.

Figure 11: Representation of the many-to-many relationship example between courses and students, using a new entity.

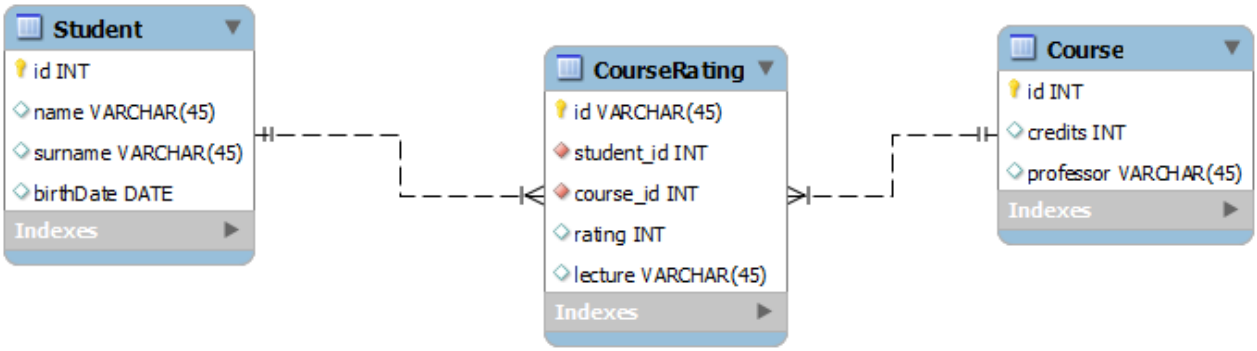

Figure 12: Implementation of the many-to-many relationship example between courses and students, using the new entity.

In this solution we only need to implement two one-to-many relatioships, one between student and course rating, and the other one between course and course rating. This is needed when, between the same student and course, there could be more than one element. That's why the lecture attribute is introduced: each student, in this example, can rate single lectures from courses.

```
//Student.java

import javax.persistence.*;
import java.time.LocalDateTime;
import java.util.Set;

@Entity
@Table(name = "student")
public class Student {

    @Id
    private long id;
    @OneToMany(mappedBy = "student")
    private Set<CourseRating> ratings;
    private String name;
    private String surname;
    private LocalDateTime birthDate;
    //getters and setters
}
```

```
//Course.java

import javax.persistence.Entity;
import javax.persistence.Id;
import javax.persistence.OneToMany;
```

```java
import java.util.Set;

@Entity
public class Course {
    @Id
    private long id;

    @OneToMany(mappedBy = "course")
    private Set<CourseRating> ratings;

    private int credits;
    private String professor;
    //getters and setters
}
```

```java
//CourseRating.java

import javax.persistence.*;

@Entity
public class CourseRating {
    @Id
    private long id;

    @ManyToOne
    @JoinColumn(name="student_id")
    private Student student;

    @ManyToOne
    @JoinColumn(name="course_id")
    private Course course;

    private int lecture;

    private int rating;
    //getters and setters
}
```

We used the merge method in all the entity manager classes (student, course and course rating) to avoid problems and orders in inserting.

```java
//StudentEM.java

import javax.persistence.EntityManager;
import javax.persistence.EntityManagerFactory;
import javax.persistence.Persistence;

public class StudentEM {
    EntityManagerFactory factory;
    EntityManager entityManager;

    public void setup(){
        factory = Persistence.createEntityManagerFactory("university");
    }

    public void exit(){
        factory.close();
```

```
    }

    public long create(Student student){
        long id=0;
        try{
            entityManager = factory.createEntityManager();
            entityManager.getTransaction().begin();
            entityManager.merge(student);
            entityManager.getTransaction().commit();
            id = student.getId();
        } catch (Exception e){
            e.printStackTrace();
        } finally {
            entityManager.close();
        }
        return id;
    }
}
```

```
//CourseEM.java

import javax.persistence.EntityManager;
import javax.persistence.EntityManagerFactory;
import javax.persistence.Persistence;

public class CourseEM {
    EntityManagerFactory factory;
    EntityManager entityManager;

    public void setup(){
        factory = Persistence.createEntityManagerFactory("university");
    }

    public void exit(){
        factory.close();
    }

    public void create(Course course){
        try{
            entityManager = factory.createEntityManager();
            entityManager.getTransaction().begin();
            entityManager.merge(course);
            entityManager.getTransaction().commit();
        } catch (Exception e){
            e.printStackTrace();
        } finally {
            entityManager.close();
        }
    }
}
```

```
//CourseRatingEM.java

import javax.persistence.EntityManager;
import javax.persistence.EntityManagerFactory;
import javax.persistence.Persistence;
```

```
public class CourseRatingEM {
    EntityManagerFactory factory;
    EntityManager entityManager;

    public void setup(){
        factory = Persistence.createEntityManagerFactory("university");
    }

    public void exit(){
        factory.close();
    }

    public void create(CourseRating rating){
        try{
            entityManager = factory.createEntityManager();
            entityManager.getTransaction().begin();
            entityManager.merge(rating);
            entityManager.getTransaction().commit();
        } catch (Exception e){
            e.printStackTrace();
        } finally {
            entityManager.close();
        }
    }
}
```

5 UPDATING

To update an object by using his primary key, we have to use the find function, defined as `T find(Class<T> entityClass, Object primaryKey)`.

Example:

To update the name of a student (defined in chapter 1) we can insert this function inside the StudentEM class

```
public void updateName(long id, String name){
    try{
        entityManager =  factory.createEntityManager();
        entityManager.getTransaction().begin();
        Student student = entityManager.find(Student.class, id);
        student.setName(name);
        entityManager.getTransaction().commit();
    } catch (Exception e){
        e.printStackTrace();
    } finally {
        entityManager.close();
    }
}
```

To update more rows at the same time, we need to use JPQL.

6 DELETING

To delete an object we need to do the same steps of the update: retrieve an object by using the function find and then, instead of using the set functions, we call `remove()`.

Example:

To remove a student, we need to insert this method in the StudentEM class:

```
public void remove (long id){
    try{
        entityManager =  factory.createEntityManager();
        entityManager.getTransaction().begin();
        Student student = entityManager.find(Student.class, id);
        if (student != null)
            entityManager.remove(student);
        entityManager.getTransaction().commit();
    } catch (Exception e){
        e.printStackTrace();
    } finally {
        entityManager.close();
    }
}
```

7 JPQL

To execute a query, we need to make a transaction using the entity manager. In particular, in this transaction we have to create a Query object created from the query string and then insert the results inside a list.

Example:

Using the student entity (chapter 1) to get all the names inside the database, we can insert this function inside StudentEM class:

```
public void showNames(){
    try{
        entityManager = factory.createEntityManager();
        entityManager.getTransaction().begin();
        Query query = entityManager.createQuery("SELECT s.name FROM Student s");
        List<String> names = query.getResultList();
        for (String s:names){
            System.out.println(s+"\n");
        }
        entityManager.getTransaction().commit();
    } catch(Exception e){
        e.printStackTrace();
    } finally {
        entityManager.close();
    }
}
```

We can also get only one result by using the function `getFirstResult()` instead of `getResultList()`. This could be useful with aggregation queries.

8 INHERITANCE

There are four possible approaches in JPA/Hibernate to achieve entities' inheritance. Let's suppose that we have a situation like the one in the following figure.

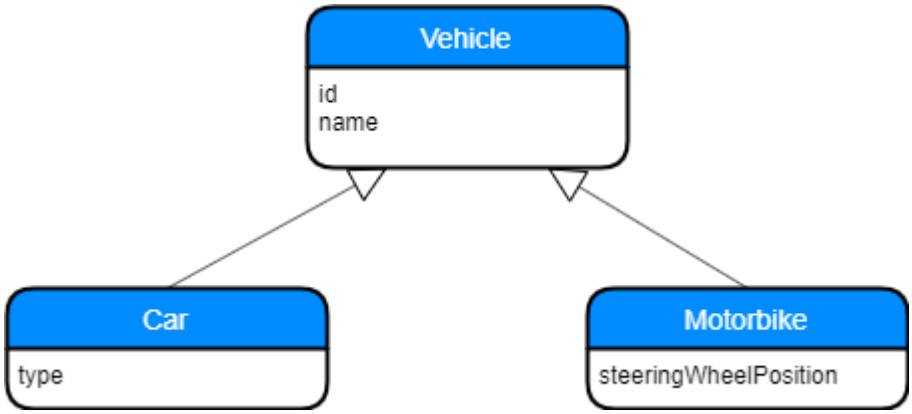

Figure 13: Entity-Relationship diagram about vehicles inheritance example.

Car and Motorbike entities are derived by Vehicle.

8.1 MAPPED SUPERCLASS

With this approach, we have to create an abstract class without @Entity annotation but in which we can insert the right annotations to the attributes (like @Id for the primary key). We have also to add a `@MappedSuperclass` annotation. This class will be extended by the derived entities, to share the same attributes. The problem of this approach is that the abstract class is not an entity, so it's impossible to use queries on it. If, for example, we have a relationship between the superclass and another object, this relationship will be implemented for each derived table.

Example:

If we consider the example made at the beginning of chapter 8 about vehicles, we have that Vehicle is the abstract Superclass with the `@MappedSuperclass` annotation.

```
//Vehicle.java

import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.MappedSuperclass;

@MappedSuperclass
public abstract class Vehicle {
    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private long id;
    private String name;
}
```

Then, we have that Car and Motorbike are the classes that extend Vehicle.

```java
//Car.java

import javax.persistence.Column;
import javax.persistence.Entity;

@Entity(name = "Car")
public class Car extends Vehicle {
    @Column(name = "wheel")
    private String steeringWheelPosition;
}
```

```java
//Motorbike.java

import javax.persistence.Column;
import javax.persistence.Entity;

@Entity(name = "motorbike")
public class Motorbike extends Vehicle {
    @Column(name = "type")
    private String type;
}
```

8.2 Table per class

"Table per class" approach is very similar to the "Mapped Superclass" approach, but the difference is that the class that will be extended is an entity. This method resolves the query problem: with it is possible to make queries that consider all the objects of the derived classes. But the problem is that queries made using this method are often complicated and performance are not so good. The superclass still has not its own database table, but has all the relationship implemented. To use this approach, we have to put an `@Entity` and `@Inheritance(strategy = InheritanceType.TABLE_PER_CLASS)` annotations on the superclass. The superclass still remains an abstract class, as we have seen in the "Mapped Superclass" approach.

Example:

Using the vehicles example, we have a code very similar to the "Mapped Superclass" approach. Car and Motorbike classes are the same. Vehicle class becomes an entity.

```java
//Vehicle.java

import javax.persistence.*;

@Entity
@Inheritance(strategy = InheritanceType.TABLE_PER_CLASS)
public abstract class Vehicle {
    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private long id;
    private String name;
}
```

8.3 SINGLE TABLE

In this approach, all entities derived from the superclass are mapped in the same table in the database. This solution is the best for query complexity and for performance. The superclass is still abstract. It is an entity and it has a `@Inheritance(strategy = InheritanceType.SINGLE_TABLE)` and a `@DiscriminatorColumn` annotations. The Discriminator Column is used to distinguish the objects type inside the table. Derived classes need to have a `@DiscriminatorValue` annotation.

Example:

In the vehicle example, we have to put the `@DiscriminatorColumn` in the Vehicle class and the `@DiscriminatorValue` in the Car and Motorbike classes. Also, we have to indicate the Single Table approach in the Vehicle class inheritance type.

```
//Vehicle.java

import javax.persistence.*;

@Entity
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn(name = "vehicleType")
public abstract class Vehicle {
    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private long id;
    private String name;
}
```

```
//Car.java

import javax.persistence.Column;
import javax.persistence.DiscriminatorValue;
import javax.persistence.Entity;

@Entity(name = "Car")
@DiscriminatorValue("Car")
public class Car extends Vehicle {
    @Column(name = "wheel")
    private String steeringWheelPosition;
}
```

```
//Motorbike.java

import javax.persistence.Column;
import javax.persistence.DiscriminatorValue;
import javax.persistence.Entity;

@Entity(name = "motorbike")
@DiscriminatorValue("Motorbike")
public class Motorbike extends Vehicle {
    @Column(name = "type")
    private String type;
}
```

8.4 JOINED

In the joined approach each class has its own database table. This seems similar to "Table per class" approach, but using "Joined" approach we have also a database table for the superclass. It's called "Joined" because each subclass requires a join between two tables to retrieve all the attributes in a query. To implement it, we need to use a `@Inheritance(strategy = InheritanceType.JOINED)` annotation in the superclass.

Example:

In the vehicle example, we need to use the `@Inheritance(strategy = InheritanceType.JOINED)` in the Vehicle class. There is nothing to change in the Car and Motorbike tables.

```
//Vehicle.java

import javax.persistence.*;

@Entity
@Inheritance(strategy = InheritanceType.JOINED)
public abstract class Vehicle {
    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private long id;
    private String name;
}
```

```
//Car.java

import javax.persistence.Column;
import javax.persistence.Entity;

@Entity(name = "Car")
public class Car extends Vehicle {
    @Column(name = "wheel")
    private String steeringWheelPosition;
}
```

```
//Motorbike.java

import javax.persistence.Column;
import javax.persistence.Entity;

@Entity(name = "motorbike")
public class Motorbike extends Vehicle {
    @Column(name = "type")
    private String type;
}
```